Architecture Orientée Services

Chapitre 1 : Introduction aux systèmes basés sur le Web

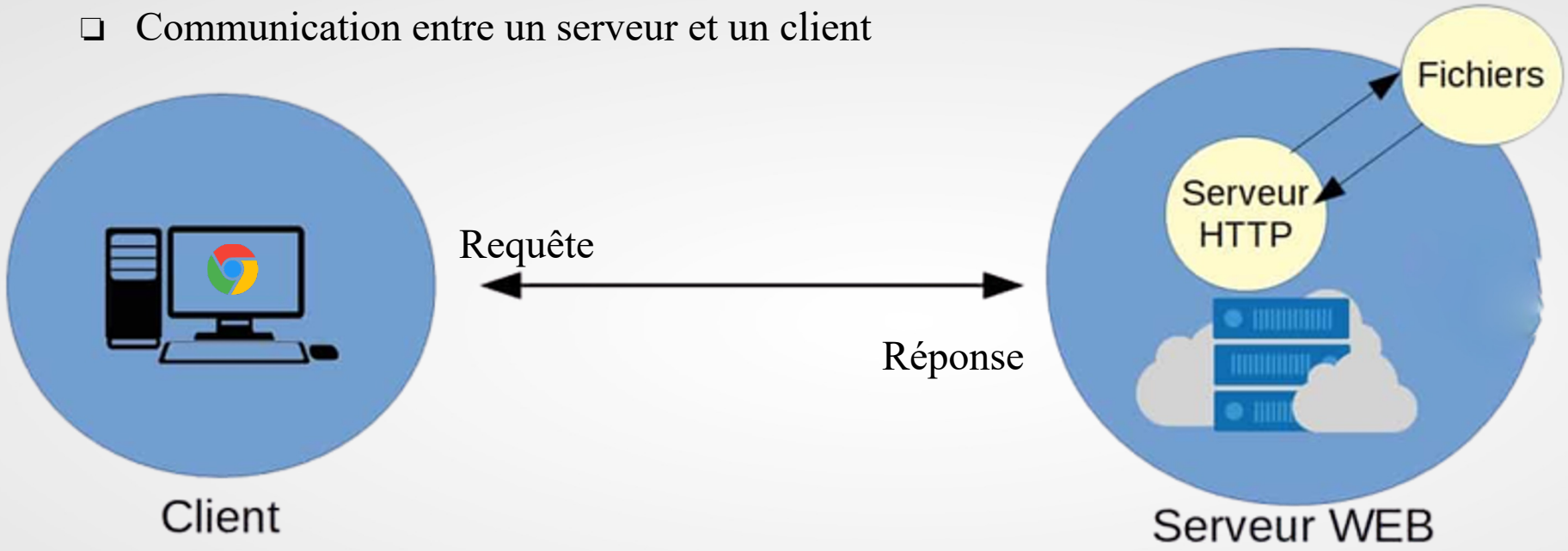
Année universitaire : 2023-2024

Plan

- Fonctionnement du Web
- Historique d'évolution du Web
- Technologies liées au Web
- Contexte de communication
- Paradigmes des architectures logicielles
- Le terme Service
- Middlewares
- Service Web
- Principes d'un service Web
- Avantages des services web

Fonctionnement du Web

- Communication entre un serveur et un client



- Le Web est désormais indispensable pour accéder à l'information, établir des communications et interagir avec divers services en ligne à l'échelle mondiale.

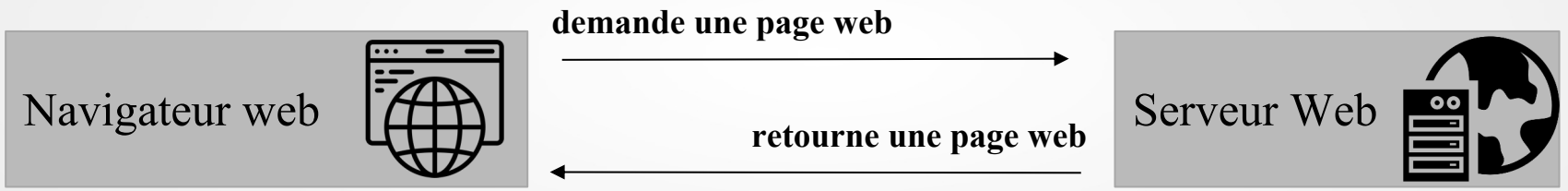
Historique d'évolution du Web

Page web static

1990

Les pages web static sont stockées sur le serveur sous la forme d'un fichier HTML distinct.

- Une page web statique affiche le même code HTML stocké sur le serveur, sans personnalisatio préalable, et les mises à jour nécessitent des modifications manuelles du document HTM correspondant.



Page web dynamique

1993

Les pages web dynamiques sont générées au moment de l'accès. Cela signifie que la page web n'existe pas sur un serveur avant d'être générée.

Historique d’évolution du Web

Applications web

Au milieu des années 1990

Les applications web peuvent fonctionner sur n'importe quel système d'exploitation, à condition qu'un navigateur web compatible soit installé.
Cela signifie qu'elles sont indépendantes de la plate-forme.

Service Web

2000

Le terme **service web** fait référence à une technologie permettant à des applications de communiquer entre elles via le Web en utilisant **des protocoles standardisés et des normes ouvertes** telles que HTTP, XML et JSON, qui sont faciles à manipuler par les machines..

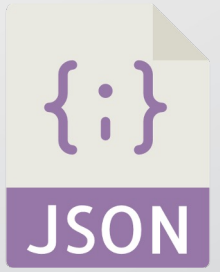


Technologies liées au Web

- Les technologies liées au Web comprennent des formats de données tels que HTML, XML et JSON pour structurer et échanger des informations.
- Le transfert de ces données est réalisé via les protocoles HTTP et TCP, utilisant des URI/URL pour localiser les ressources.
- Les requêtes et réponses HTTP sont organisées selon une structure spécifique, et les méthodes HTTP permettent d'effectuer différentes actions sur les ressources du Web.



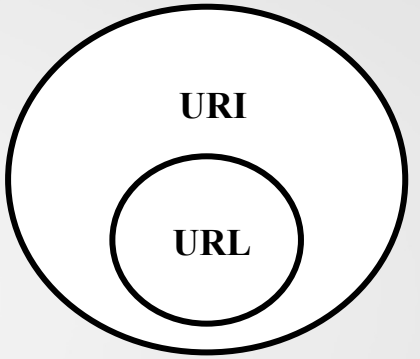




Technologies liées au Web

URI/URL

- **URI** (Uniform Resource Identifier) : C'est un identifiant unique utilisé pour identifier une ressource de manière générique.
- **URL** (Uniform Resource Locator) : C'est une forme spécifique d'URI qui indique l'emplacement exact d'une ressource sur le Web.



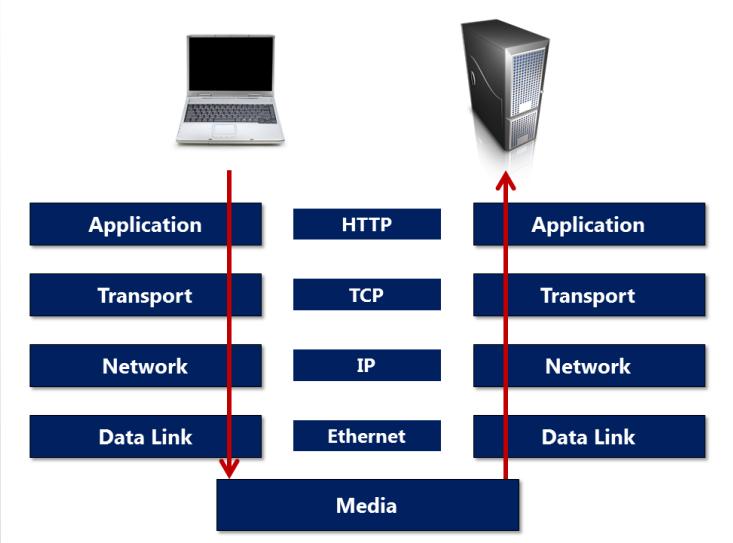
Remarque :

L'URI n'a pas besoin de fournir explicitement l'adresse IP. Le navigateur est capable de résoudre l'adresse IP correspondant au nom d'hôte fourni dans l'URI.
Le navigateur va soit connaissent déjà l'adresse IP correspondant à le nom d'hôte, ou s'il ne le sait pas, il interrogera un Serveur DNS (Domain Name System) pour le savoir.

Technologies liées au Web

TCP/HTTP

- **TCP** (Transmission Control Protocol) est un protocole de la couche de transport qui assure une communication fiable et orientée connexion entre les appareils sur un réseau.
- **HTTP** (Hypertext Transfer Protocol) est un protocole de la couche application qui s'exécute au-dessus de TCP (ou d'autres protocoles de la couche de transport) et régit le transfert de ressources hypertexte (web) entre les clients et les serveurs sur le World Wide Web.



Contexte de communication

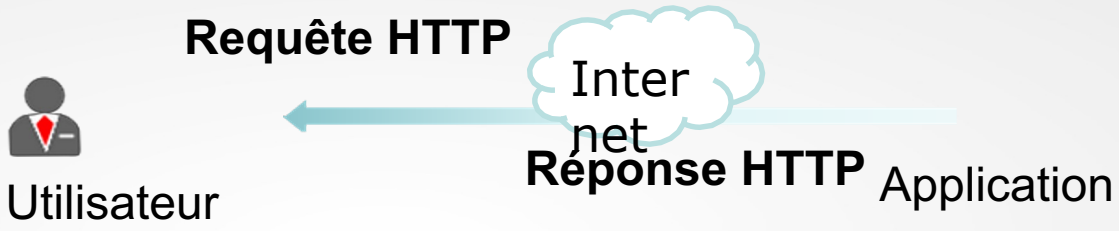
Human-centric web

- Le Web centré utilisateur implique que l'humain est l'acteur principal

pour l'initialisation de l'ensemble des requêtes

B2C

Business ToConsumer



Application-centric web

- Le Web centré application a pour objectif de permettre à des applications de différentes organisations de communiquer entre elles

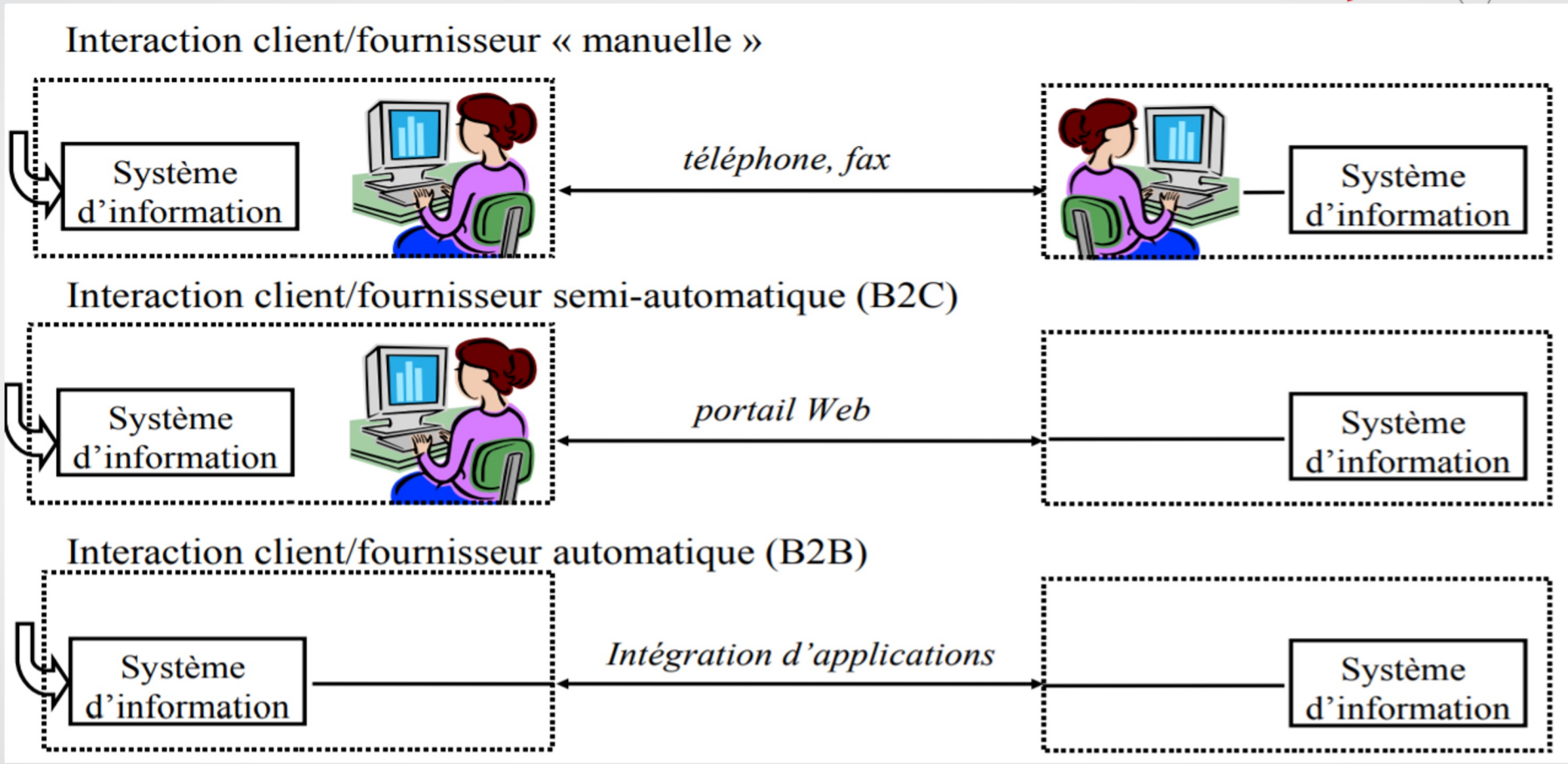
B2B

Business To Business



Contexte de communication

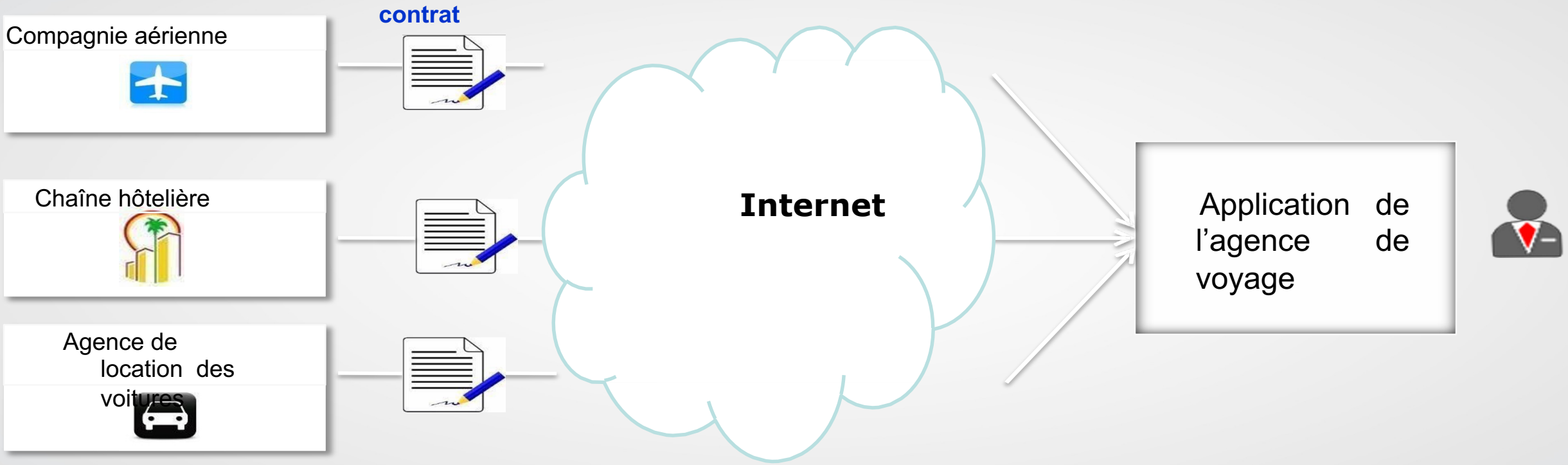
Historique



Source: https://www.ibisc.univ-evry.fr/~tmelliti/cours/CPAR/cours6.pdf

Contexte de communication

Exemple B2B

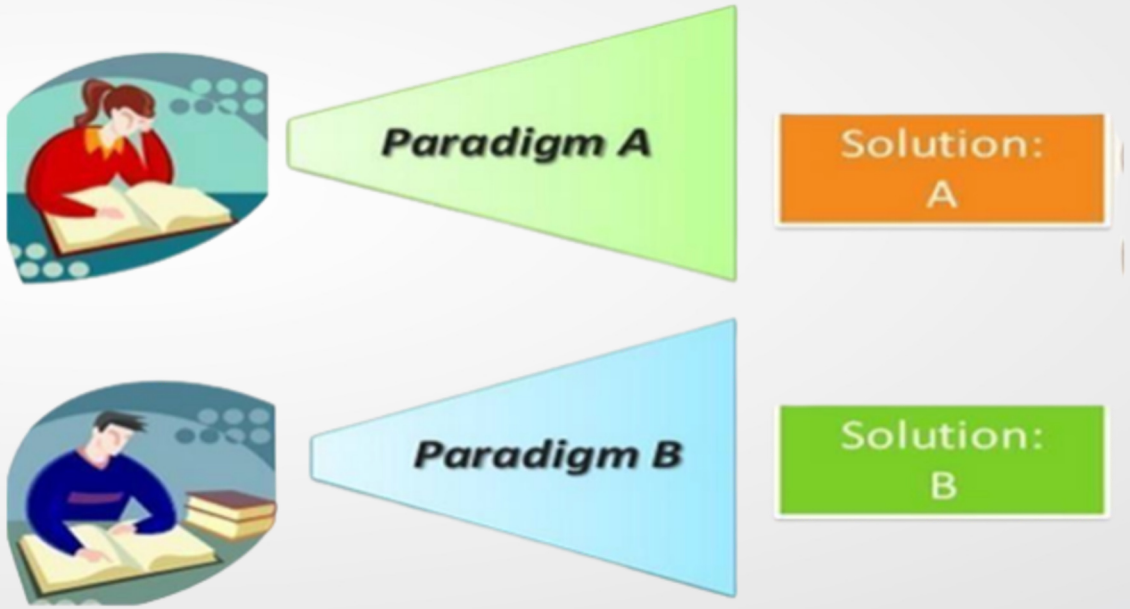


Paradigmes des architectures logicielles

Le terme de paradigme est employé pour exprimer la façon dont un système a été conçu et pensé dans ses grandes lignes.

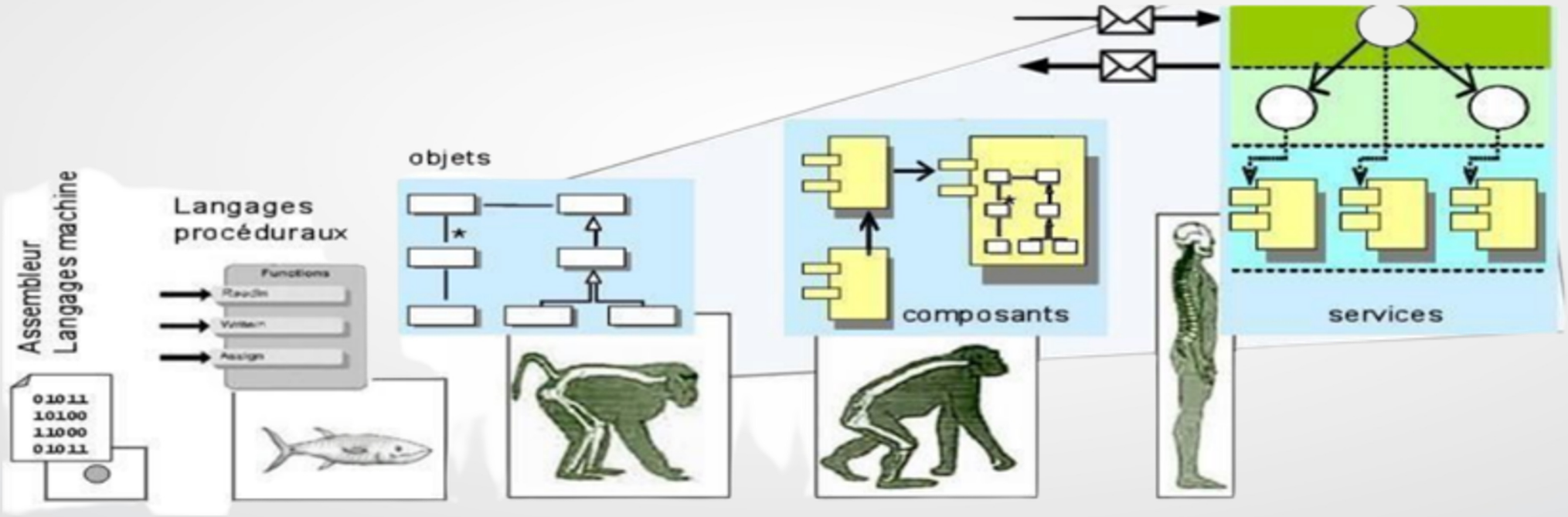


Objectif:
Développer une application de facturation



Paradigmes des architectures logicielles

- Les révolutions informatiques coïncident généralement avec un changement de paradigme.
- Niveau d'abstraction grandissant avec l'évolution des paradigmes



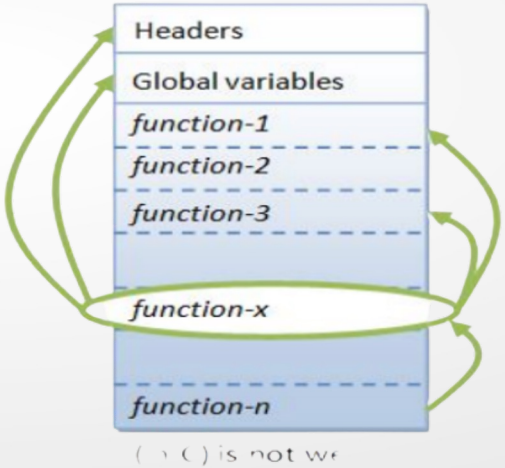
Paradigmes des architectures logicielles

Paradigme procédural

- le code est organisé en procédures (ou fonctions) qui contiennent des séquences d'instructions. Ces procédures sont appelées successivement pour accomplir une tâche spécifique.
- Un Système informatique **désordonné**

Limites :

- Tend à générer du code "Spaghetti"
- Maintenance complexe
- Modularité et abstraction absente
- Réutilisation ardue



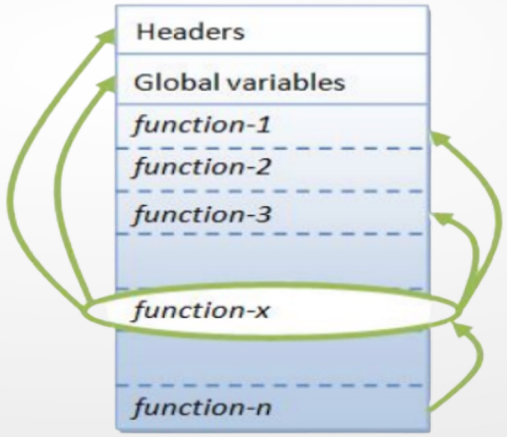
Paradigmes des architectures logicielles

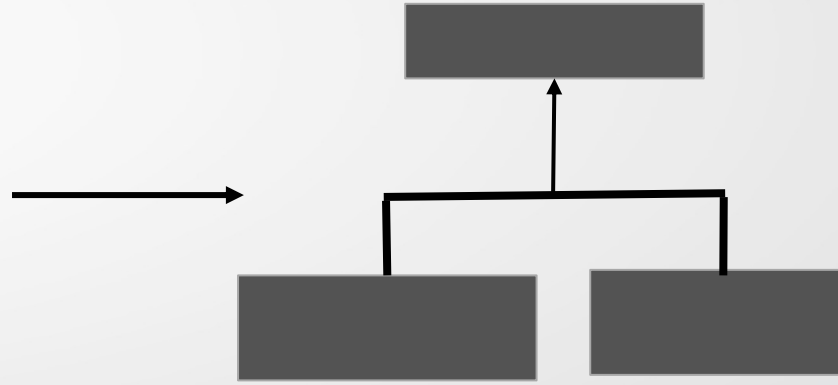
Paradigme objet

- L'idée est de concevoir les programmes non plus comme des lignes de codes qui s'exécutent séquentiellement, mais comme des objets qui dialoguent
- Ses principes incluent l'abstraction encapsulation,de données, polymorphisme et héritage.

Limites :

- Réutilisation difficile
- Couplage fort
- Problème de maintenance





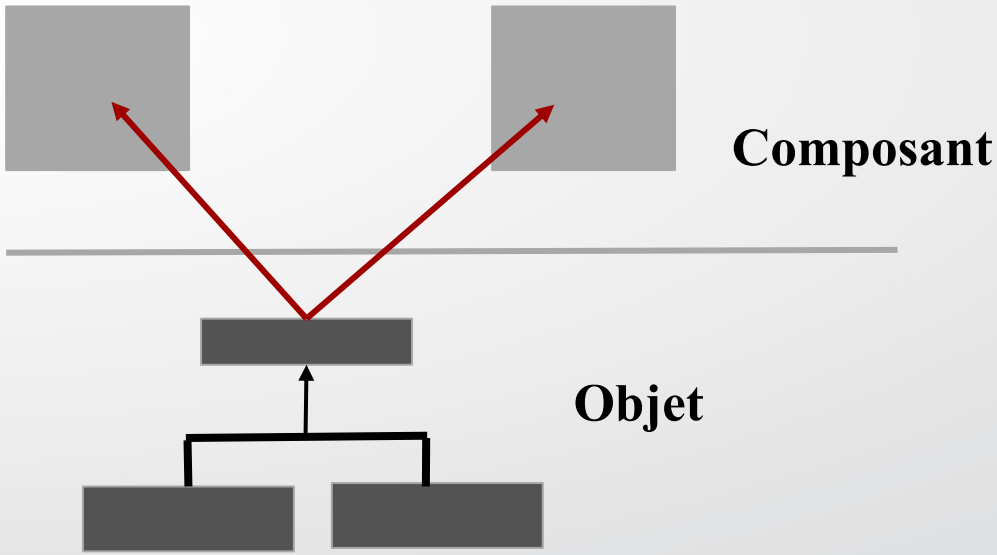
Paradigmes des architectures logicielles

Paradigme composant

- Construire une application composée par un ensemble de briques de base configurables et réutilisables.
- Il s'agit d'externaliser le code fonctionnel d'une application afin de le rendre réutilisable dans d'autres applications

Limites :

- Complexité de gestion des dépendances
- Interopérabilité entre composants hétérogènes



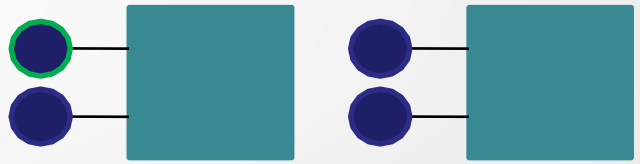
Paradigmes des architectures logicielles

Paradigme service

- Prise en charge de la diversité et de l'hétérogénéité des termes de langages systèmes de programmation, et de logiciels, en de technologies de conception (et de réalisation) ou de plates-formes d'exécution

- Le paradigme service permet de:

- réduire le couplage
- améliorer la réutilisation
- augmenter l'abstraction





Service Composant Objet

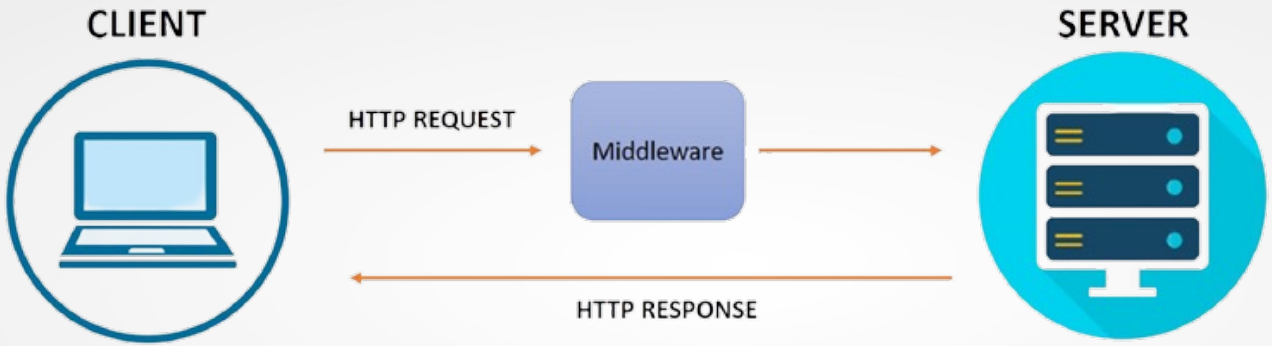
Le terme Service

- Un service est un terme général utilisé en informatique pour décrire une unité autonome de fonctionnalités qui peut être exécutée de manière indépendante

- Les services peuvent être à la fois locaux et distants. Les services locaux s'exécutent sur la même machine que l'application qui les consomme, tandis que les services distants peuvent s'exécuter sur des machines différentes ou être hébergés dans le cloud.

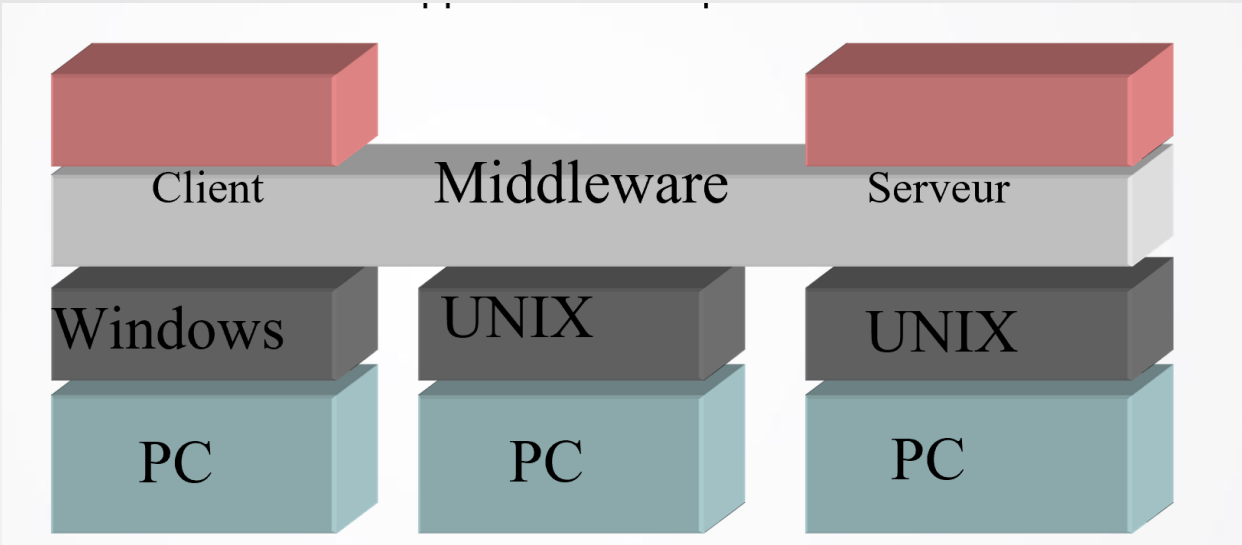
Middlewares : intergiciel

Un logiciel servant d'intermédiaire entre d'autres logiciels
Un intermédiaire de communication entre des applications complexes et distribuées



Le middleware joue un rôle essentiel en simplifiant la communication et l'intégration entre applications, services ou composants logiciels dans un environnement distribué. Il agit comme une couche d'abstraction, permettant aux différents éléments du système de travailler harmonieusement ensemble, indépendamment de leurs différences techniques, langages de programmation ou plates-formes de développement.

Middlewares : intergiciel



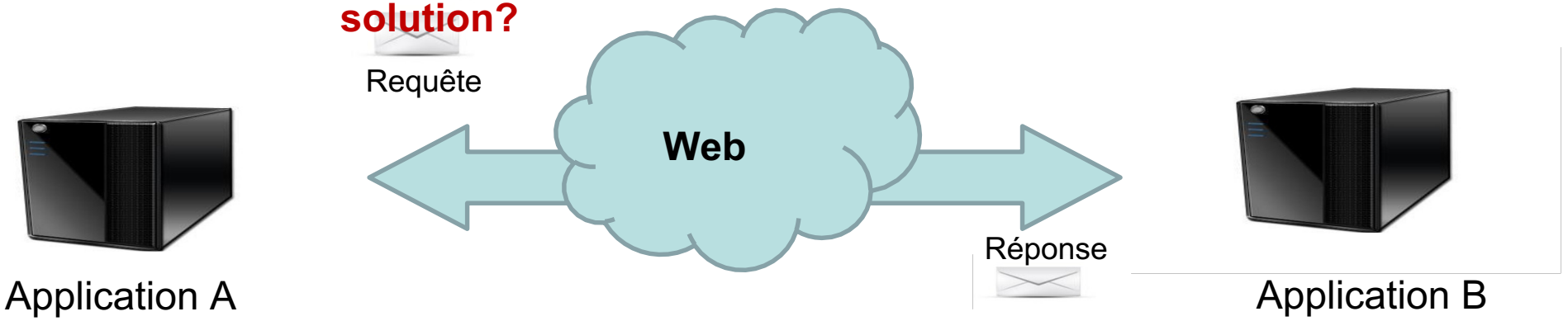
Rôles de base d'un middleware:

- Résoudre l'**intéropérabilité** : Unifier l'accès à des machines distantes
- Résoudre l'**hétérogénéité** : Etre indépendant des systèmes d'exploitation et du langage de programmatio des applications

Middlewares : intergiciel

- Solutions existantes:
  - DCOM,
  - .NET Remoting
    - Middlware .Net
  - RMI (Remote Method Invocation)
    - Middlware Java permet de faire communiquer des *objets java* **distribués** sur le réseau
  - CORBA (Common Object Request Broker Architecture)
    - Permet de faire communiquer des objets écrits dans des langages différents (C++, Java, Smalltalk)
  -
  -

Quels sont les atouts d'une meilleure solution?



Services Web

- Service web = **service + web**
- Un service Web met à disposition un service **via Internet**

- Un SW est un **programme informatique**, permettant la communication et l'échange de données entre applications et systèmes hétérogènes dans des environnements distribués.
- Les services Web interagissent à travers l'échange de messages

Services Web

Il constitue ainsi une interface permettant à deux machines (ou applications) de communiquer. Pour y parvenir, la technologie doit disposer de deux propriétés essentielles :

être multiplateforme : il n'est pas nécessaire que le client et le serveur aient la même configuration pour pouvoir communiquer. Le service Web leur permet de se retrouver à un même niveau.

être partagée : dans la plupart des cas, un service Web est à disposition de plus d'un client. Différents clients accèdent à ce service via Internet.

➔ **Lorsqu'un service Web est utilisé, un client envoie une requête à un serveur et déclenche ainsi une action auprès de ce serveur. Le serveur renvoie ensuite une réponse au client.**

Services Web

❑Il existe deux grandes familles de services web:

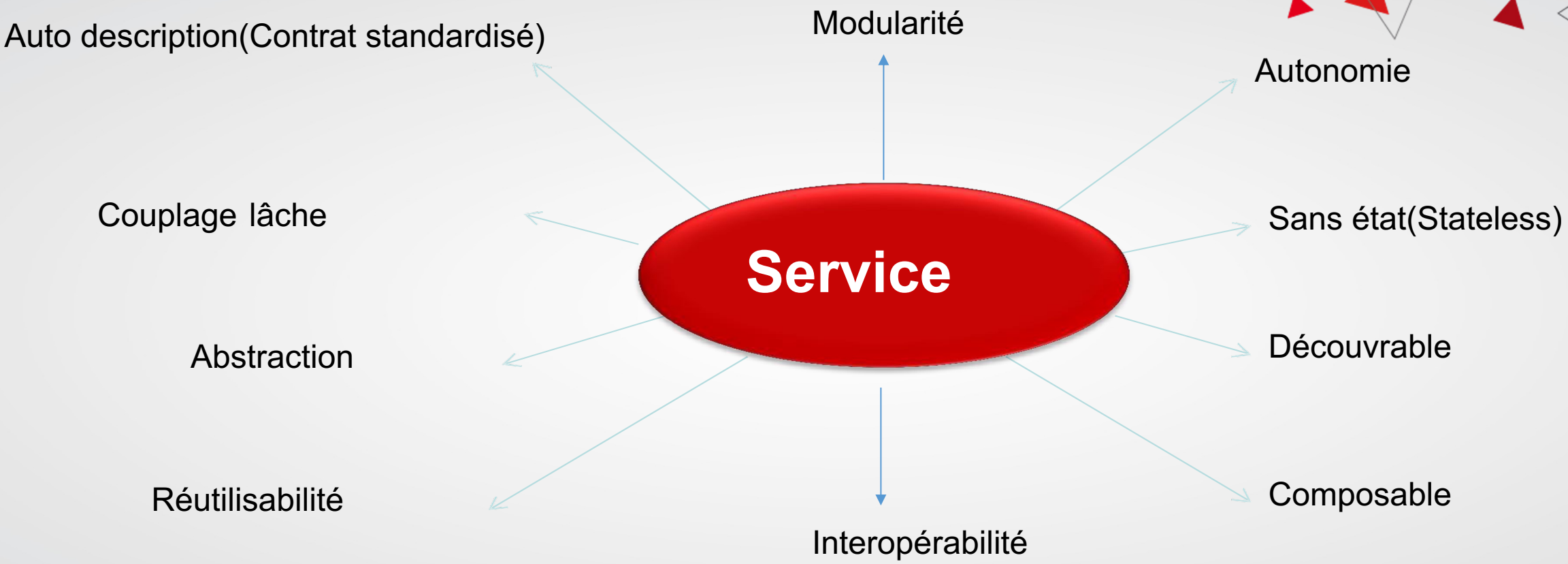
- Les services web **étendus** utilisant les standards:
  - SOAP pour la communication ;
  - UDDI (annuaire) pour la publication ;
  - WSDL (contrat) pour la description

- Les services web **REST** utilisant :
  - Directement HTTP au lieu d'une enveloppe SOAP ;
  - Un URI pour nommer et identifier une ressource ;
  - Les méthodes HTTP (POST, GET, PUT et DELETE) pour effectuer les opérations de base CRUD

Services Web : REST / SOAP

	REST	SOAP
Structure de réponse	Fixé par le serveur	Fixé par le serveur
Format de données	JSON, XML, TXT, HTML	XML
P. Transport	HTTP	HTTP, SMTP, FTP
P. Communication	HTTP	SOAP
Méthodes	GET, POST, PUT, DELETE	Noms opérations
Contrat standarisé	WADL (optionnel)	WSDL (obligatoire)

Principes d'un service Web



Principes d’un service Web

Auto description (Contrat standardisé)

L’ensemble des services d’un même Système Technique sont exposés au travers de contrats respectant les mêmes règles de standardisation.

Abstraction

Le service fonctionne en « **boîte noire** »

- Seul le contrat du service (informations nécessaires pour l’invocation) est exposé au consommateur du service .

- Le fonctionnement interne du service (sa logique métier et son implémentation) ne sont **pas visibles** .

Principes d'un service Web

Couplage lâche

- Dépendance faible entre le consommateur et le service
  - **Dépendance du contrat** et non pas de l'implémentation;
  - Echange à travers des **messages;**
  - Orchestration qui assure l'indépendance des services vu qu'elle leur permet de communiquer pour réaliser un processus, sans avoir à se connaître

Principes d’un service Web

Autonomie

- Un service ne doit être dépendant d'aucun contexte ou service externe:
  - Son comportement est indépendant du contexte fonctionnel et technique dans lequel il a été invoqué.

Stateless

- Un service ne stocke pas les informations des clients:
  - Ne stocke pas de données;
  - Ne fait référence à aucune transaction passée.

Principes d'un service Web

Composable

Un service peut participer à des compositions de services

- Un ensemble de services peuvent être composés à travers leur orchestration pour répondre à un besoin complexe;
- **Avantage:**
  - Apport de valeur ajoutée (répondre à un nouveau besoin complexe)

Principes d'un service Web

Modularité

La modularité est matérialisée par le fait que les différents services sont indépendants les uns des autres dans la plateforme de services et peuvent être accessibles par les applications distantes tournant dans des environnements hétérogènes. D'où la notion d'interopérabilité.

Interopérabilité

Réfère à la capacité d'un système à coexister et à coopérer avec d'autres systèmes éventuellement **hétérogènes** selon un schéma ouvert d'interconnexion.

Langages de programmation différents,
SGBD différents,
Architectures différentes,
etc.

Principes d’un service Web

- Intégration vs Interopérabilité –

- **Integration** means that you've writtensome custom code to connect two (or more) systems together.
  - *[Bobby Wolf – IBMArchitect]*

- Lorsqu'on parle d'intégration, nous pensons au processus qui fait que différents systèmes d'information **apparaissent comme un seul**.

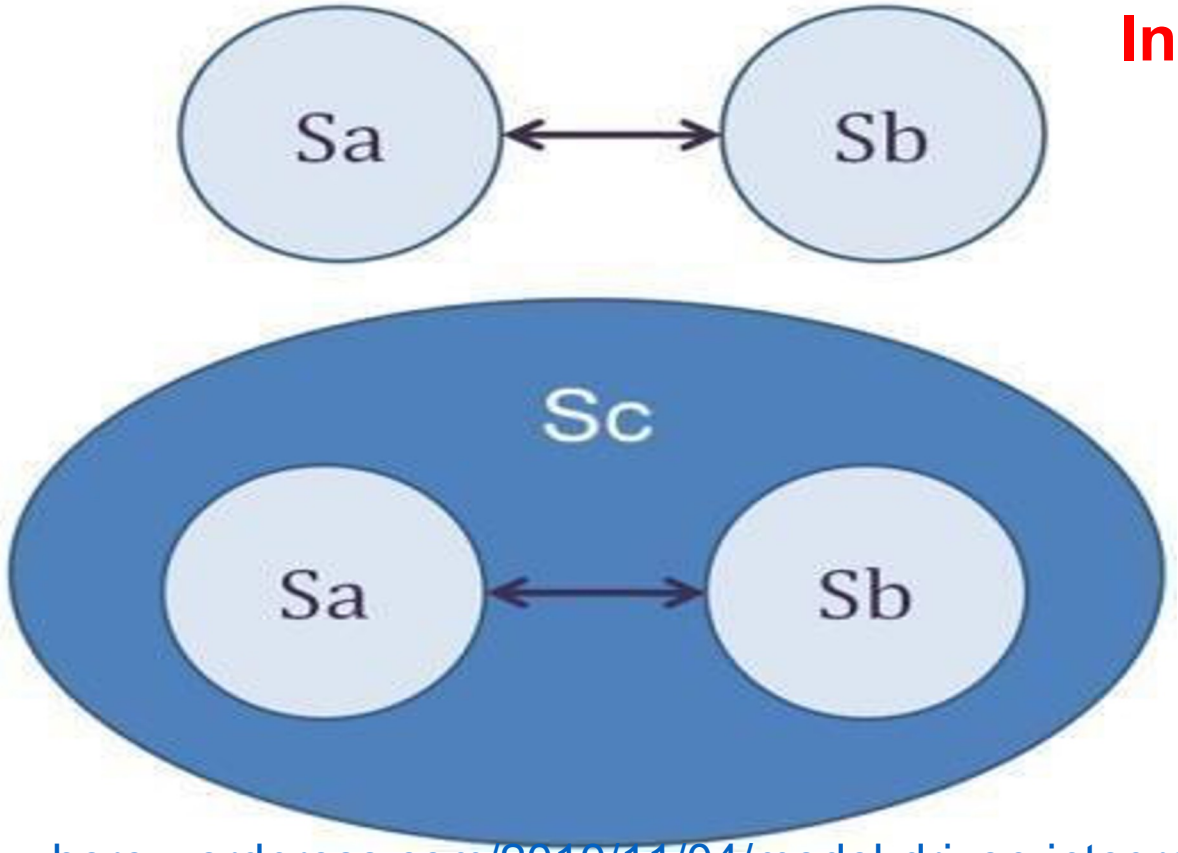
Principes d'un service Web

- Intégration vs Interopérabilité –

Two heterogeneous Systems Sa and Sb

One « integrated » system Sc

(Does not mean that Sa and Sb have been « merged »)



Interoperabilité

Integration

Source: http://modelseverywhere.wordpress.com/2010/11/04/model-driven-integration/

Avantages des services web

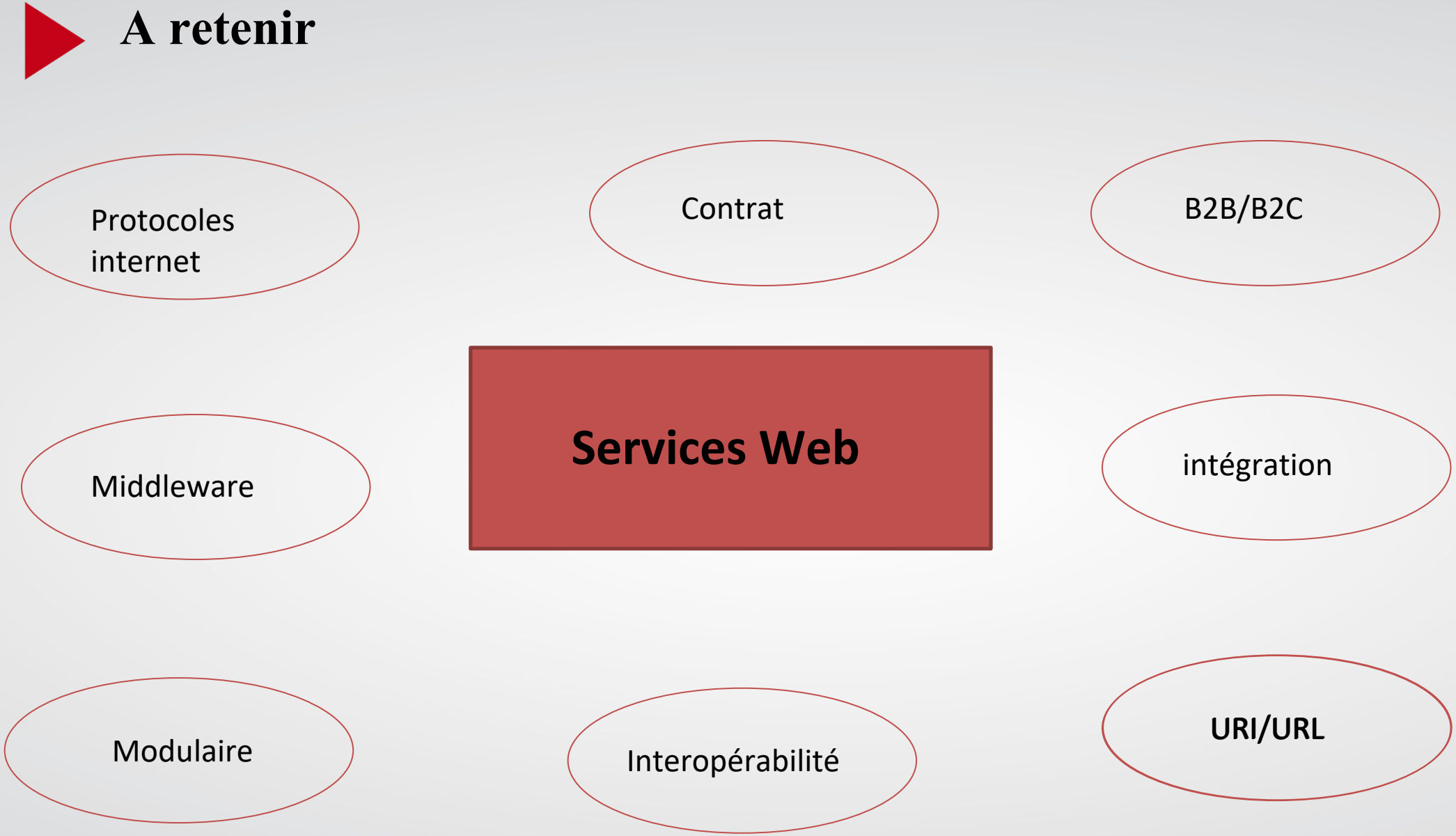
- Offrir une technologie adaptée aux applications B2B;

- Rendre possible et plus facile l'interconnexion et l'interaction des systèmes et composants hétérogènes;
- Utilisés par le Web Sémantique pas seulement le web interactif

- Garantir l'interopérabilité et donner lieu à des systèmes plus ouverts que ceux utilisant des protocoles tels que RPC, DCOM, RMI… ;
- Réutilisables dans un environnement ouvert;

- Garantir un couplage lâche.

A retenir


Protocoles internet
Contrat
B2B/B2C
Services Web
Middleware
intégration
Modulaire
Interopérabilité
URI/URL

Références

http://fr.wikipedia.org/wiki/Paradigme
https://www.coursera.org/learn/service-oriented-architecture/

http://design-patterns.fr/introduction-a-la-_programmation-orientee-objet

http://fr.wikipedia.org/wiki/Middleware

http://blog.xebia.fr/2009/04/29/soa-du-composant-_au-service-lautonomie

https://www.ibisc.univ-evry.fr/~tmelliti/cours/CPAR/cours6.pdf

http://adslbox.free.fr/rapports/rapport-gl-service-oriented-architecture.pdf

http://deptinfo.unice.fr/~baude/WS/cours_SOA_AO+FB.pdf